# Interrupts

ECE 2564, Embedded Systems

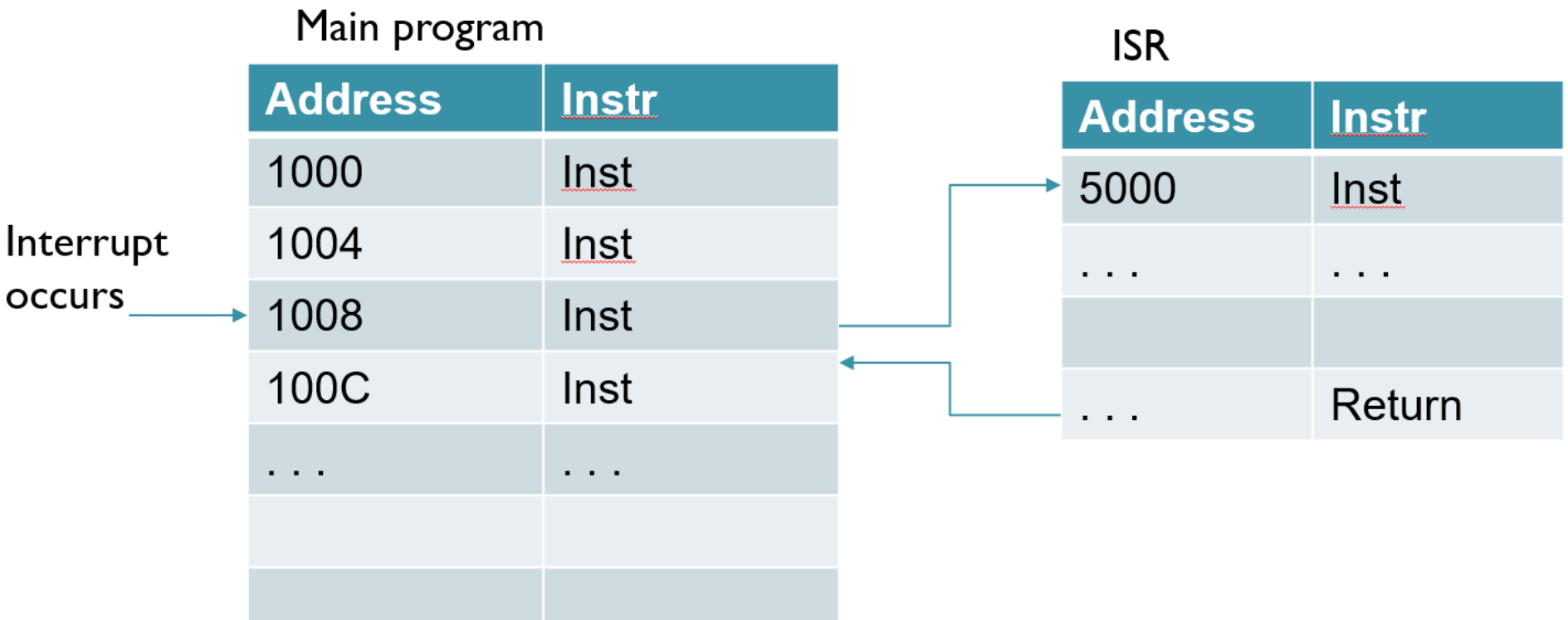
Fall 2025

# Overview

- This module will provide code demonstrations of interrupts
- Download **this** repository for the interrupt examples code

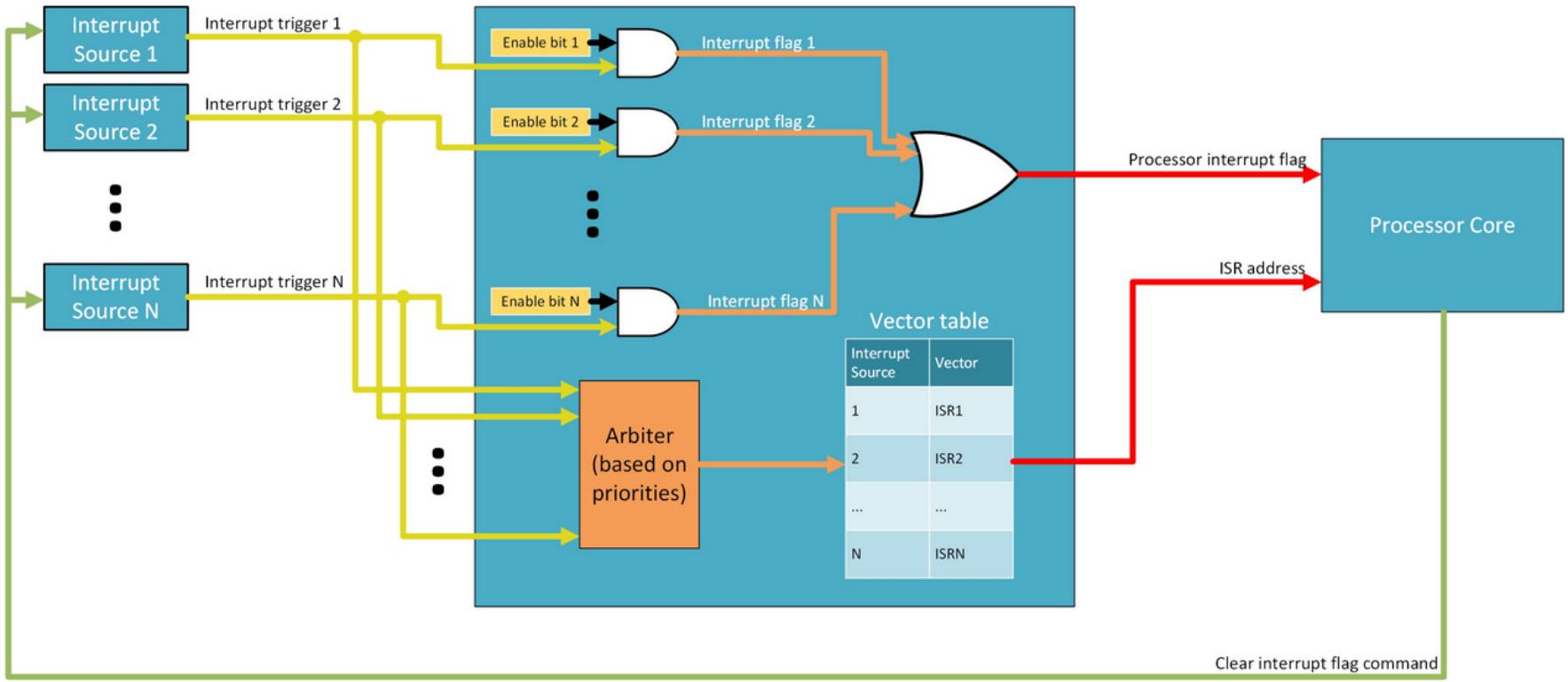
# Recall: Interrupts

- Polling versus interrupt-driven code
- How do interrupts work?
- What is an Interrupt Service Routine (ISR)?
- How to *program* them?



# Recall: Interrupt Controller

- The programmer decides which interrupts are enabled
- The Interrupt Controller sends *both* the interrupt flag and the ISR address to the CPU
- Priority of interrupts decided by the arbiter



# Interrupts from Peripherals

- Interrupt trigger:
- A hardware signal from a peripheral device, requesting service
- Examples:
  - A transition on a GPIO pin
  - Timer32 expired
  - Timer_A CCR0 rollover
  - Timer_A counter becomes equal to CCRn (compare condition is met)
- Interrupt enable:
  - A bit in the Interrupt Controller that allows the interrupt trigger to pass
- Interrupt flag:
  - A bit that indicates when a valid (enabled) interrupt request has occurred
- Interrupt priorities:
  - A set of variables that decide which interrupt has priority over another
- Interrupt vector:
  - The address of the first instruction of an ISR
- Interrupt vector table:
  - An architectural entity that stores all the interrupt vectors

# Four Steps

- Step 0: clear the interrupt flag during the initialization
- Step 1: set up the interrupt source
- Step 2: enable the interrupt in the interrupt controller (NVIC)
- Step 3: set up the ISR in the vector table and write the ISR
  - within the ISR, make sure you tell the source that the interrupts has been serviced, i.e., acknowledge the interrupt
- Step 4: write the main function that interfaces with the ISR

# Programming Steps

- Step 0: Make sure to *clear* the interrupt flag prior to enabling it
- Step 1: As part of the initialization of the peripheral make it possible for the *interrupt source* to trigger an interrupt
  - This is creating the possibility that the yellow wires in the architecture figure become 1 occasionally

```
void initLB1() {
    ...
    GPIO_clearInterruptFlag(GPIO_PORT_P1, GPIO_PIN1);

    // enable interrupt on port 1, pin 1
    GPIO_enableInterrupt(GPIO_PORT_P1, GPIO_PIN1);

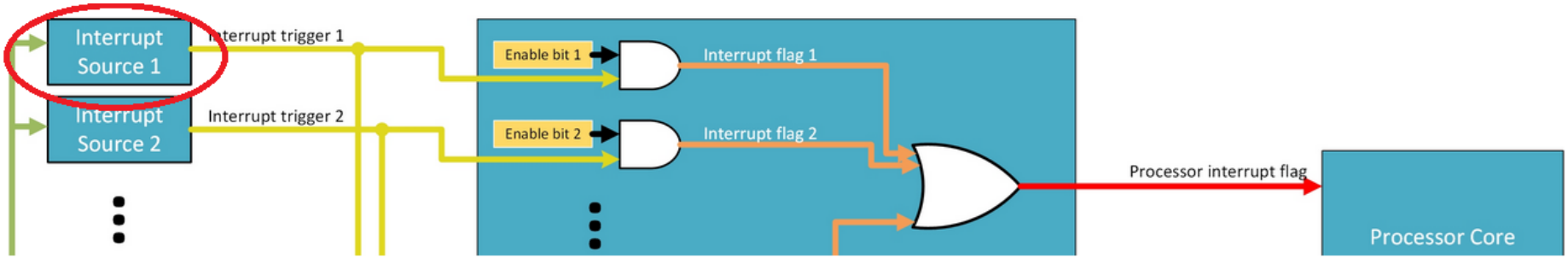
    ...
}
```

# Programming Steps: Step 1 (Continued)

- We need to talk to the interrupt *source* — the peripheral such as GPIO, UART, ...
  - Some peripherals automatically trigger an interrupt (Timer32 triggers an interrupt when the timer expires)
  - Some peripherals need a specific command for them to be able to trigger an interrupt (GPIO for example)
- Step 1.1: Set the conditions under which the peripheral should trigger an interrupt (this is peripheral dependent)
  - Example: Tell GPIO to trigger an interrupt when the pin goes from high to low

```
void initLB1() {
    ...
    // the interrupt is triggered on high to low transition (tapping)
    GPIO_interruptEdgeSelect(GPIO_PORT_P1, GPIO_PIN1,
                             GPIO_HIGH_TO_LOW_TRANSITION);
    ...
}
```

# Programming Steps




```
void initLB1() {
    // Initializing S1 on Pin1 of Port 1
    GPIO_setAsInputPinWithPullUpResistor (GPIO_PORT_P1, GPIO_PIN1);

    GPIO_clearInterruptFlag(GPIO_PORT_P1, GPIO_PIN1);  // Step 0

    // enable interrupt on port 1, pin 1                                    //
    GPIO_enableInterrupt(GPIO_PORT_P1, GPIO_PIN1);                          //
                                                                            // Step 1
    // the interrupt is triggered on high to low transition (tapping)       //
    GPIO_interruptEdgeSelect(GPIO_PORT_P1, GPIO_PIN1,                       //
                             GPIO_HIGH_TO_LOW_TRANSITION);                  //

    // enable the port 1 interrupt
    Interrupt_enableInterrupt(INT_PORT1);                  // Step 2
}
```

# Programming Steps

- Step 2: Enable the interrupt in the interrupt controller
  - To do this we talk to the interrupt controller

```
// enable the port 1 interrupt
Interrupt_enableInterrupt(INT_PORT1);
```

  - NOTE: This is different from the `GPIO_enableInterrupt()`
- We are allowing the orange interrupt flag signals in the architecture figure to become 1 when needed
  - Each interrupt source is associated with a certain register in the interrupt controller
  - Certain macros in driverlib help us figure out what source is associate with what enable register

# External Interrupts[1]

- There are 57 external interrupts
- There are interrupts for Timer_A, Timer32, ports, and many more

```
108 #define INT_PORT1                       (51) /* PORT1 IRQ */
109 #define INT_PORT2                       (52) /* PORT2 IRQ */
110 #define INT_PORT3                       (53) /* PORT3 IRQ */
111 #define INT_PORT4                       (54) /* PORT4 IRQ */
112 #define INT_PORT5                       (55) /* PORT5 IRQ */
113 #define INT_PORT6                       (56) /* PORT6 IRQ */
114 #define INT_LCD_F                       (57) /* PORT6 IRQ */
115
116 #define NUM_INTERRUPTS                  (57)
```

[1] From `interrupt.h` included in `driverlib.h`

# Timer32 Interrupts

```
98 #define INT_T32_INT1                (41) /* T32_INT1 IRQ */
99 #define INT_T32_INT2                (42) /* T32_INT2 IRQ */
```

- Recall: We have two Timer32 timers
  - `Timer32_0` and `Timer32_1`
- To enable Timer32 interrupts we refer to
  - `INT_T32_INT1` and `INT_T32_INT2`
    - `Timer32_0` is associated with `INT_T32_INT1`
    - `Timer32_1` is associated with `INT_T32_INT2`
- There are inconsistencies in the MSP432
- We need to pay careful attention to the user manual and data sheets

# Device Specific Interrupt Handler Names[2]

```
/* device specific interrupt handler */
...
extern void T32_INT1_IRQHandler (void) __attribute__((weak,alias("Default_Handler")
extern void T32_INT2_IRQHandler (void) __attribute__((weak,alias("Default_Handler")
...
extern void PORT1_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
extern void PORT2_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
extern void PORT3_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
extern void PORT4_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
extern void PORT5_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
extern void PORT6_IRQHandler    (void) __attribute__((weak,alias("Default_Handler")
```

[2] From startup_msp432p401r_ccs.c

# Programming Steps

Step 3: Write the ISR

- Step 3.1: We have to identify the correct name for the ISR or register the name
  - Finding the name
    - Interrupt trigger $\longrightarrow$ interrupt macro $\longrightarrow$ interrupt service routine
    - Example: P1.1 $\longrightarrow$ `INT_PORT1` $\longrightarrow$ `PORT1_IRQHandler`
  - Registering the ISR
    - We can also use any name we wish and register the function as an ISR
    - In this case the registration is part of initialization
  - Step 3.2: We must use the proper `volatile global` variable to communicate with the rest of the code
  - Step 3.3: We must clear the interrupt
    - Tell the interrupt source that the interrupt has been serviced so that the interrupt source clears the interrupt

# Programming the Port1 IRQ Handler

```
// The ISR for port 1 (all of port 1, not any specific pin).
// We did not choose the name of this function. Any time a
// Port 1 interrupt occurs this ISR is called automatically.
void PORT1_IRQHandler() {

    // Only S1, attached to Pin1, can provide the interrupt.
    // However, this is not true for all programs, so in
    // general, you have to check which pin created the interrupt
    if (GPIO_getInterruptStatus(GPIO_PORT_P1, GPIO_PIN1))
        S1modifiedFlag = true;

    // The very critical step to make sure once we leave ISR, we don't
    // come back to ISR again.
    // This tells the GPIO, the CPU has heard the interrupt and
    // it should clear it.
    GPIO_clearInterruptFlag(GPIO_PORT_P1, GPIO_PIN1);
}
```

# Notes on Volatile Global Variables

- *volatile* is used in embedded software development to inform the compiler that the value of the variable may be changed by hardware
  - This definition prevents the compiler from optimizing global, shared variables
  - In this sample code we use the volatile global variable named `S1modifiedFlag`

# Programming Steps

- Step 4: Write the body of the ISR
  - Step 4.1: Initialize the global variables
  - Step 4.2: Use the global variables to make use of the interrupts

  - NOTE: the LaunchPad green LED indicates that the CPU is sleeping

# The Body of the ISR Illustrated

```
int main(void)
{   initialize();
    while (1) {
        // Enters the Low Power Mode 0 - the processor is
        // asleep and only responds to interrupts
        TurnOn_LLG();  // go into low power
        PCM_gotoLPM0();
        TurnOff_LLG();

        if (S1modifiedFlag) {
            TurnOn_LL1();
            // We must reset our flag so that we don't think
            // there has been an interrupt the next time we enter the loop
            S1modifiedFlag = false;

            // start the 2-second timer configured to give interrupts
            Timer32_setCount(TIMER32_0_BASE, TIMER_WAIT);
            Timer32_startTimer(TIMER32_0_BASE, true);
        }
        if (TimerExpiredFlag) {
            TurnOff_LL1();

            // Reset our timer flag so we don't think the timer recently expired.
            TimerExpiredFlag = false;
        }}}
```

# Advantages of Interrupts

- The CPU can spend much of its time in low-power or sleep mode
- The CPU wakes up only in response to an interrupt
- When polling the CPU can't go into low-power or sleep mode most of the time because it must constantly poll to see if something needs attention

- Caution: ISR and user code communicate using global variables

# Interrupts for Timer32

- Some of the instructions in Step 1 are not needed
  - The default for timer interrupt is already set
- For instructional purposes we will name the ISR function
  - We aren't using the default ISR function name
  - We'll need to *bind* the function name, (`registerInterrupt`)

```
void initTimer(){
    // Initialize the timer 0
    Timer32_initModule(TIMER32_0_BASE,
                       TIMER32_PRESCALER_1,
                       TIMER32_32BIT,
                       TIMER32_PERIODIC_MODE);

    // register the TimerExpired() function as the ISR for Timer32_0
    Timer32_registerInterrupt(INT_T32_INT1, TimerExpired);

    Timer32_clearInterruptFlag(TIMER32_0_BASE);

    Interrupt_enableInterrupt(INT_T32_INT1);
}
```

# ISRs for Timer32

- Using the global variable `TimerExpiredFlag` to communicate between the ISR and main code

```
// A boolean variable that is true when Timer32 is expired
volatile bool TimerExpiredFlag = false;
```

- Remember to clear the interrupt flag

```
void TimerExpired()  // another ISR
{
    // We use this global variable to communicate with the main
    TimerExpiredFlag = true;

    // We tell the Timer32 to remove the interrupt as we
    // already handled it.
    Timer32_clearInterruptFlag(TIMER32_0_BASE);
}
```

# Choreographing the GPIO and Timer32 ISRs

- `PCM_gotoLPM0()` places the CPU in low power mode
- The processor is asleep and can be awoken in response to interrupts, such as from a GPIO or a Timer32 interrupt
- We'll see in the code that the CPU goes to sleep again after servicing an interrupt

# Choreographing the GPIO and Timer32 ISRs

```
int main(void)
{   initialize();
    while (1) {
        // Enters the Low Power Mode 0 - the processor is
        // asleep and only responds to interrupts
        TurnOn_LLG();  // go into low power
        PCM_gotoLPM0(); // <--- ISR is run upon CPU wakeup; code resumes from here
        TurnOff_LLG();

        if (S1modifiedFlag) { // <--- this flag is set in GPIO ISR
            TurnOn_LL1();
            // We must reset our flag so that we don't think
            // there has been an interrupt the next time we enter the loop
            S1modifiedFlag = false;

            // start the 2-second timer configured to give interrupts
            Timer32_setCount(TIMER32_0_BASE, TIMER_WAIT); // <--- timer setup here
            Timer32_startTimer(TIMER32_0_BASE, true);
        }
        if (TimerExpiredFlag) { // <--- this flag is set in Timer32 ISR
            TurnOff_LL1();

            // Reset our timer flag so we don't think the timer recently expired.
            TimerExpiredFlag = false;
        }}}
```

# Interrupt-based Versus Polling-based Programming

| | Interrupt-based | Polling-based |
|---|---|---|
| Use of global variables? | Yes | Not necessary |
| Quick response? | Fast | Could be slow |
| Structure of the code? Ease of debugging and testing | Harder to write & debug | Easier to write & debug |
| Low-power? | Yes | No |

- Use interrupt-based for
  - High-priority inputs
  - Low-pwer needs
- Use polling-based for everything else

# Debugging Code in an ISR

- It may be difficult to trigger an interrupt while single-stepping through the code
- You may have to set a breakpoint in the ISR and trigger the interrupt in real-time
- Once you have hit a breakpoint within an ISR you can single-step inside the ISR
- Monitor the global variables
- Use LEDs to signal status or write diagnostic messages to the display

# Summary

- We learned how to write interrupt-driven code
  - We explored examples for the GPIO and Timer32
  - We practiced the four steps for successful interrupt programming